

Metodi Numerici per l'Analisi dei Dati

Assegnata una funzione $f(x)$ reale di variabile reale sviluppare una funzione in python per la risoluzione dell'equazione non lineare:

$$f(x) = 0$$

Di seguito uno snippet di codice da trasformare in una funzione Python

- Considerato il metodo di Dekker-brent come variante del metodo di Bisezione, si sviluppi una funzione python che lo implementi.

```

comment mca 2310;
boolean procedure zeroin(x, y, fx, tol); real x, y, fx, tol;
begin real a, fa, b, fb, c, fc, tol, m, p, q;
  a:= x; fa:= fx; b:= x:= y; fb:= fx;
  interpolate: c:= a; fc:= fa;
  extrapolate: if abs(fc) < abs(fb) then
    begin a:= b; fa:= fb; x:= b:= c; fb:= fc; c:= a; fc:= fa
    end interchange;
  tol:= tol; m:= (c + b) × .5; if abs(m - b) > tol then
  begin p:= (b - a) × fb; if p ≥ 0 then q:= fa - fb else
    begin q:= fb - fa; p:= - p end;
    a:= b; fa:= fb;
    x:= b:= if p < abs(q) × tol then sign(c - b) × tol + b else
      if p < (m - b) × q then p / q + b else m; fb:= fx;
    goto if sign(fb) = sign(fc) then interpolate else
      extrapolate
    end;
  y:= c; zeroin:= sign(fb) × sign(fc) ≤ 0
end zeroin;

```

Dekker-Brent ovvero procedura zeroin implementazione originale del 1969

- Sia $f(x)$ una funzione a valori vettoriali si implementi una funzione python che prenda in ingresso lo Jacobinao e resitutisca una approssimazione dello zero.

Newton's method generalizes naturally to multiple dimensions. We seek a solution $\mathbf{x}$ for

$$f(\mathbf{x}) = \mathbf{0} \quad [2.149]$$

where $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$ is differentiable. Start with seed value $\mathbf{x}^{[1]}$, and recursively obtain subsequent values $\mathbf{x}^{[2]}, \mathbf{x}^{[3]}, \mathbf{x}^{[4]}, \dots$ as follows. At each value $\mathbf{x}^{[k]}$, construct Jacobian approximation $\tilde{f}^{[k]}$ for f :

$$\tilde{f}^{[k]}(\mathbf{x}) = f(\mathbf{x}^{[k]}) + Jf(\mathbf{x}^{[k]})(\mathbf{x} - \mathbf{x}^{[k]}) \quad [2.150]$$

and select $\mathbf{x}^{[k+1]}$ as the solution to

$$\tilde{f}^{[k]}(\mathbf{x}) = \mathbf{0} \quad [2.151]$$

Substituting [2.250] into [2.251] and setting $\mathbf{x} = \mathbf{x}^{[k+1]}$ yields the recursive equation

$$\mathbf{x}^{[k+1]} = \mathbf{x}^{[k]} - [Jf(\mathbf{x}^{[k]})]^{-1} f(\mathbf{x}^{[k]}) \quad [2.152]$$

which generalizes [2.142]